USBD43: Add a new Electric locomotive model - Sprint 3

Este documento descreve a implementação, configuração de dados e testes realizados para a **USBD43**.

1. Descrição da User Story

- As a Planner, I want to add a new Electric locomotive model

Critérios de Aceitação e Regras de Negócio:

- Os campos obrigatórios não podem ser NULL ou vazios (ID, nome, fabricante, descrição da bitola).
- Os valores numéricos devem ser positivos (potência kW, peso toneladas, velocidade máxima km/h, tração kN).
- O sistema deve impedir a duplicação de IDs (chave primária).
- O sistema deve impedir a duplicação de nomes, ignorando diferenças de maiúsculas/minúsculas e espaços extra (normalização).
- Específico para elétrico: tensão, frequência.
- A implementação deve ser feita através de uma função PL/SQL.

---

2. Explicação do Script (Função PL/SQL)

A lógica central da **USBD43** foi encapsulada na função PL/SQL `fn_create_electric_loco_model`. Esta abordagem permite centralizar as regras de negócio e garantir a integridade dos dados, independentemente de onde a inserção é solicitada.

Detalhes da Implementação:

1. **Assinatura da Função:** A função aceita múltiplos parâmetros: `p_model_id` (ID), `p_model_name` (Nome), `p_make` (Fabricante), valores numéricos (potência, peso, etc.), `p_voltage` (Tensão), `p_frequency` (Frequência), `p_gauge_desc` (Descrição da Bitola), dimensões e bogies.
2. **Validação de Dados Obrigatórios:** Verifica se ID, nome, fabricante e descrição da bitola são NULL/vazios (lança erro -20003).
3. **Validação de Valores Numéricos:** Verifica se valores como potência, peso, velocidade e tração são positivos (lança erro -20003).
4. **Verificação de Duplicados (ID):** Consulta a tabela `Locomotive_Model` para verificar se o ID já existe (lança erro -20004).
5. **Normalização de Texto:** Para o nome, remove espaços extras e converte para maiúsculas para comparação.
6. **Verificação de Duplicados (Nome):** Consulta a tabela comparando o nome normalizado (lança erro -20001).
7. **Validação da Bitola:** Busca o valor da bitola pela descrição; se não encontrado, lança erro -20002.
8. **Inserção Segura:** Executa o `INSERT` com engine_type 'Electric' e COMMIT em sucesso.
9. **Transação em Erro:** Em caso de exceção, executa `ROLLBACK` e relança o erro.

Código: `USBD43-script.sql`

```
CREATE OR REPLACE FUNCTION fn_create_electric_loco_model(
    p_model_id      IN NUMBER,
    p_model_name    IN VARCHAR2,
    p_make          IN VARCHAR2,
    p_power_kw      IN NUMBER,
    p_weight_tons   IN NUMBER,
    p_max_speed_kmh IN NUMBER,
    p_traction_kn   IN NUMBER,
    p_voltage       IN VARCHAR2,
    p_frequency     IN VARCHAR2,
    p_gauge_desc    IN VARCHAR2,
    p_length_m      IN NUMBER,
    p_width_m       IN NUMBER,
    p_height_m      IN NUMBER,
    p_num_bogies    IN NUMBER,
    p_bogies_type   IN VARCHAR2
) RETURN NUMBER IS
    v_count         NUMBER;
    v_gauge_val     NUMBER;
    v_clean_name    VARCHAR2(100);
    v_norm_name     VARCHAR2(100);

BEGIN
    -- 1. Validação de Inputs Obrigatórios
    IF p_model_id IS NULL OR TRIM(p_model_name) IS NULL OR TRIM(p_make) IS NULL OR TRIM(p_gauge_desc) IS NULL THEN
        RAISE_APPLICATION_ERROR(-20003, 'Mandatory fields cannot be null or empty.');
END IF;

    -- 2. Validar valores numéricos (Devem ser positivos)
    IF p_power_kw <= 0 OR p_weight_tons <= 0 OR p_max_speed_kmh <= 0 OR p_traction_kn <= 0 THEN
        RAISE_APPLICATION_ERROR(-20003, 'Numeric values must be positive.');
```

```
END IF;

    -- 3. Verificar se o ID já existe
SELECT COUNT(*) INTO v_count FROM Locomotive_Model WHERE id = p_model_id;
IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20004, 'Model ID already exists.');
END IF;

    -- 4. Normalização e Verificação de Nome Duplicado (Case/Space Insensitive)
    v_clean_name := REGEXP_REPLACE(TRIM(p_model_name), '\s+', ' ');
    v_norm_name  := UPPER(v_clean_name);

SELECT COUNT(*) INTO v_count FROM Locomotive_Model
WHERE UPPER(REGEXP_REPLACE(TRIM(model_name), '\s+', ' ')) = v_norm_name;

IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Locomotive model name already exists.');
END IF;

    -- 5. Validar Bitola (Gauge)
BEGIN
SELECT gauge_value INTO v_gauge_val FROM Railway_Gauge WHERE description = TRIM(p_gauge_desc);
EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20002, 'Gauge description not found.');
END;

    -- 6. Inserção
INSERT INTO Locomotive_Model (
    id, model_name, make, power_kw, weight_tons, max_speed_kmh, traction_kn,
    engine_type, voltage, frequency, gauge, length_m, width_m, height_m,
    num_bogies, bogies_type
) VALUES (
             p_model_id, v_clean_name, TRIM(p_make), p_power_kw, p_weight_tons,
             p_max_speed_kmh, p_traction_kn, 'Electric', TRIM(p_voltage), TRIM(p_frequency),
             v_gauge_val, p_length_m, p_width_m, p_height_m, p_num_bogies, TRIM(p_bogies_type)
         );

-- 7. Gestão de Transação: Sucesso
COMMIT;
RETURN p_model_id;

EXCEPTION
    WHEN OTHERS THEN
        -- 8. Gestão de Transação: Falha
        ROLLBACK;
        RAISE;
END;
/
```

3. Explicação de Cada Teste

Para validar a User Story USBD43, foram desenvolvidos 6 cenários de teste que cobrem inserções válidas, violações de integridade e regras de normalização de dados.

Teste 1: Happy Path

Objetivo: Verificar se é possível criar um novo modelo de locomotiva elétrica válido que ainda não existe no sistema.
Descrição: Realiza limpeza preventiva, executa a função com parâmetros válidos e verifica o retorno.
Resultado Esperado: A função deve retornar o ID e o registro deve ser persistido.

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
    v_test_id NUMBER := 100;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 1: Happy Path ---');
    -- Limpeza preventiva
DELETE FROM Locomotive_Model WHERE id = v_test_id OR UPPER(model_name) = 'EUROSPRINTER ES64';
COMMIT;

v_id := fn_create_electric_loco_model(v_test_id, 'EuroSprinter ES64', 'Siemens', 6400, 87, 230, 300, '25kV', '50Hz', 'Bitola Ibérica
    DBMS_OUTPUT.PUT_LINE('Success! Created ID: ' || v_id);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Test Failed: ' || SQLERRM);
END;
/
```

Teste 2: Duplicate Name Detection

Objetivo: Verificar se o sistema normaliza nomes e impede duplicados.
Descrição: Realiza limpeza, insere um modelo base, então tenta inserir uma variação normalizada.
Resultado Esperado: A segunda inserção deve falhar com erro -20001 (duplicado).

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 2: Duplicate Name Detection ---');
    -- Limpeza
DELETE FROM Locomotive_Model WHERE id IN (200, 201) OR UPPER(model_name) = 'INDEPENDENT TEST MODEL';
COMMIT;

v_id := fn_create_electric_loco_model(200, 'Independent Test Model', 'Siemens', 6400, 87, 230, 300, '25kV', '50Hz', 'Bitola Ibérica'

BEGIN
        v_id := fn_create_electric_loco_model(201, '   INDEPENDENT    test    MODEL   ', 'Siemens', 6400, 87, 230, 300, '25kV', '50H
        DBMS_OUTPUT.PUT_LINE('Test Failed: Duplicate allowed.');
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20001 THEN
                DBMS_OUTPUT.PUT_LINE('Success: Duplicate blocked correctly.');
ELSE RAISE; END IF;
END;
END;
/
```

Teste 3: Non-Existent Gauge

Objetivo: Verificar se o sistema impede inserções com bitola inexistente.
Descrição: Tenta criar modelo com descrição de bitola inválida.
Resultado Esperado: Deve lançar erro -20002.

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 3: Non-Existent Gauge ---');
BEGIN
        v_id := fn_create_electric_loco_model(102, 'Train 3000 X', 'Alstom', 5000, 90, 120, 250, '25kV', '50Hz', 'Mars Gauge', 19.2,
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20002 THEN
                DBMS_OUTPUT.PUT_LINE('Success: Invalid Gauge blocked.');
ELSE RAISE; END IF;
END;
END;
/
```

Teste 4: Negative Numeric Values

Objetivo: Verificar se o sistema impede valores numéricos negativos.
Descrição: Tenta criar modelo com potência negativa.
Resultado Esperado: Deve lançar erro -20003.

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 4: Negative Numeric Values ---');
BEGIN
        v_id := fn_create_electric_loco_model(103, 'Weak Train', 'Alstom', -100, 90, 120, 250, '25kV', '50Hz', 'Bitola Ibérica', 19.
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20003 THEN
                DBMS_OUTPUT.PUT_LINE('Success: Negative value blocked.');
ELSE RAISE; END IF;
END;
END;
/
```

Teste 5: NULL Model Name

Objetivo: Verificar se o sistema impede inserções com nome NULL.
Descrição: Tenta criar modelo com nome NULL.
Resultado Esperado: Deve lançar erro -20003.

```
SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 5: NULL Model Name ---');
BEGIN
        v_id := fn_create_electric_loco_model(104, NULL, 'Alstom', 5000, 90, 120, 250, '25kV', '50Hz', 'Bitola Ibérica', 19.2, 3.0,
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20003 THEN
                DBMS_OUTPUT.PUT_LINE('Success: NULL input blocked.');
ELSE RAISE; END IF;
END;
END;
/
```

Teste 6: Empty/Whitespace Name

Objetivo: Verificar se o sistema impede inserções com nome vazio ou apenas espaços.
Descrição: Tenta criar modelo com nome ' '.
Resultado Esperado: Deve lançar erro -20003.

```
 SET SERVEROUTPUT ON;
DECLARE
v_id NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- TEST 6: Empty/Whitespace Name ---');
BEGIN
        v_id := fn_create_electric_loco_model(105, '   ', 'Alstom', 5000, 90, 120, 250, '25kV', '50Hz', 'Bitola Ibérica', 19.2, 3.0,
EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -20003 THEN
                DBMS_OUTPUT.PUT_LINE('Success: Empty string blocked.');
ELSE RAISE; END IF;
END;
END;
/
```